



AI / ML Engineer Interview Study Guide

Every concept behind the Stock Flag Bot project — mapped to what to study,
plus a 90+ question academic Q&A bank for AI Engineer & ML Engineer interviews

Companion to the Stock Flag Bot Documentation · July 2026

This project uses a fraction of the ML field — this guide covers the surrounding fundamentals
an interviewer will assume you know even if they weren't all in the code.

1. How to Use This Guide

This project deliberately uses **deterministic rules + one LLM**, not trained ML models — that was the right engineering choice (explained in the main documentation), but it means the codebase alone won't prepare you for an ML/AI engineer interview, where classical ML theory, deep learning fundamentals, and statistics are tested regardless of what you personally shipped.

This guide has two parts:

- **Section 2** — a map from *what's actually in this project* to *the broader concept you should study*, so you can honestly say "I used X, which is a specific case of the general idea Y" — the strongest kind of interview answer.
- **Sections 3–10** — the concepts themselves in plain language, one topic at a time.
- **Section 11** — a 90+ question academic Q&A bank, grouped by topic, answers written as tight flashcards (2–4 sentences each — enough to jog full recall, not a textbook).

Study strategy: for every concept below, be ready to (1) explain it in one plain sentence, (2) give the formula/mechanism, (3) state one real trade-off or failure mode, and (4) connect it — honestly — to something in this project or say clearly "I haven't used this, but I understand it because...". Interviewers respect precise honesty far more than vague overclaiming.

2. Concept Map — Project Feature → What To Study

What this project does	The general concept behind it	Study in section
17 signals combined by a weighted sum into one score	Linear ensembling / model stacking; weighted voting classifiers	§5 Ensembles
Signals as engine output, fed to a hypothetical future classifier	Feature engineering; feature vectors; feature importance	§4 Feature Engineering
Backtester: train/evaluate only on past data at each step	Walk-forward validation; time-series cross-validation; look-ahead bias/leakage	§4 Time Series ML
BUY/SELL/HOLD flag correctness scored later	Classification evaluation: precision/recall, confusion matrix, class imbalance	§5 Evaluation
Gemini chat grounded on computed evidence	Retrieval-augmented generation (RAG), grounding, hallucination mitigation	§7 LLMs & RAG
Gemini system prompt + evidence + history	Prompt engineering, in-context learning, context window management	§7 LLMs & RAG
Model fallback ladder on 429/503	Inference reliability, rate limiting, graceful degradation	§7, §9 MLOps
Tool-calling agents (<code>simple_agent.py</code> etc.)	Function calling, agentic AI, ReAct-style loops, multi-agent systems	§7 LLMs & Agents
Structured output (30-field Pydantic model)	Constrained decoding / structured generation, JSON-schema-guided sampling	§7 LLMs
Proposed: ML classifier over the 17 signals	Gradient boosting (XGBoost/LightGBM), decision trees, bias-variance	§3 Classical ML
Proposed: FinBERT sentiment	Transfer learning, fine-tuning a pretrained transformer, tokenization	§6 NLP, §7 LLMs
Proposed: regime detection (HMM/k-means)	Unsupervised learning, clustering, hidden Markov models	§3 Classical ML
Proposed: RL for execution	Markov decision processes, policy/value functions, PPO	§10 Reinforcement Learning
Journal + outcome scoring over weeks	Online evaluation, drift detection, A/B-test style measurement	§8 Statistics, §9 MLOps
GitHub Actions scheduled retraining-shaped pipeline	MLOps: CI/CD for ML, experiment tracking, model registries	§9 MLOps

3. Classical Machine Learning Fundamentals

3.1 The learning paradigms

- **Supervised learning:** learn a mapping $X \rightarrow y$ from labeled examples (regression = continuous y , classification = discrete y).
- **Unsupervised learning:** find structure with no labels — clustering (k-means, DBSCAN, hierarchical), dimensionality reduction (PCA, t-SNE, UMAP), anomaly detection.
- **Semi-supervised:** a little labeled data + a lot of unlabeled data.
- **Self-supervised:** labels are manufactured from the data itself (predict the next word, predict a masked token) — this is how every modern LLM is pretrained.
- **Reinforcement learning:** an agent learns by acting in an environment and receiving reward signals, not from labeled examples (see §10).

3.2 Bias–variance tradeoff (the central idea of ML)

Bias = error from overly simple assumptions (underfitting — the model can't capture the pattern even on training data). **Variance** = error from being too sensitive to the specific training sample (overfitting — great on train, poor on unseen data). Total expected error $\approx \text{bias}^2 + \text{variance} + \text{irreducible noise}$. Model complexity trades one for the other; the goal is the sweet spot, found via validation curves and regularization.

3.3 Regularization

- **L1 (Lasso):** adds $|w|$ penalty $\rightarrow$ drives some weights to exactly zero $\rightarrow$ automatic feature selection, sparse models.
- **L2 (Ridge):** adds w^2 penalty $\rightarrow$ shrinks weights smoothly, never exactly zero $\rightarrow$ handles multicollinearity well.
- **Elastic Net:** a weighted mix of both.
- **Dropout, early stopping, data augmentation** are regularization techniques specific to neural networks (see §4).

3.4 Core algorithm families to know cold

Family	Key idea	When it's the right tool
Linear/Logistic Regression	Weighted sum of features (+ sigmoid for classification)	Baseline, interpretability, linearly separable-ish data
Decision Trees	Recursive splits maximizing information gain / Gini reduction	Interpretable, handles non-linear splits, no scaling needed
Random Forest	Bagging: many trees on bootstrapped samples + random feature subsets, majority vote/average	Strong baseline, resists overfitting vs a single tree
Gradient Boosting (XGBoost/LightGBM/CatBoost)	Sequentially fit trees to the <i>residual errors</i> of prior trees	State of the art on tabular data — the natural choice for "ML over my 17 signals"
SVM	Maximize the margin between classes; kernel trick for non-linear boundaries	Smaller datasets, clear margin, high-dimensional sparse data
k-NN	Classify by the majority label of the k nearest points	Simple baseline, no training phase, sensitive to scale/curse of dimensionality
Naive Bayes	Applies Bayes' theorem assuming feature independence	Text classification, fast baselines
k-means	Iteratively assign points to nearest centroid, recompute centroids	Unsupervised grouping (e.g. market-regime clustering)
Hidden Markov Model (HMM)	Hidden states generate observed data; transitions follow Markov property	Sequential/regime data — e.g. "is the market trending or choppy"

3.5 Gradient descent

The optimization workhorse: iteratively step parameters in the direction that most reduces the loss, $\theta \leftarrow \theta - \eta \nabla L(\theta)$. Variants: **batch** (whole dataset per step, stable but slow), **stochastic (SGD)** (one sample per step, noisy but fast), **mini-batch** (the practical default). Adaptive optimizers (**Adam**, **RMSPprop**, **AdaGrad**) scale the learning rate per-parameter using gradient history — Adam is the default choice in nearly all modern deep learning.

4. Feature Engineering & Time-Series ML

4.1 Feature engineering

Turning raw data into inputs a model can learn from. In this project, the 17 strategy signals *are* engineered features (direction, strength, timeframe) — exactly the shape a tabular ML model wants. General techniques: scaling/normalization (StandardScaler, MinMax), encoding categoricals (one-hot, target/frequency encoding), interaction terms, binning, and domain-specific derived features (which is what technical indicators like RSI/MACD/ATR actually are — engineered features on top of raw OHLCV).

4.2 Feature importance & explainability

SHAP (Shapley Additive Explanations, from cooperative game theory) attributes a prediction's difference from the average to each feature fairly and consistently — the standard tool for "why did the model say BUY". **LIME** explains individual predictions by fitting a simple local surrogate model around that one prediction. Tree-based models also expose built-in feature importance (split count / gain) — cheaper but less rigorous than SHAP.

4.3 Time-series-specific pitfalls (the most commonly missed topic)

- **Look-ahead bias / leakage:** using information that wouldn't have been available at prediction time. The single most common way people accidentally produce a fake winning strategy.
- **Why random k-fold cross-validation is wrong for time series:** it can train on the future and test on the past. Use **walk-forward validation** (a.k.a. rolling-origin evaluation): train on $[0, t]$, test on $(t, t+k]$, then roll t forward and repeat — exactly what this project's backtester does.
- **Stationarity:** a stationary series has constant mean/variance over time; most price series aren't (they trend), so many models work on *returns* or differenced series instead of raw price. The **Augmented Dickey-Fuller test** checks for stationarity.
- **Autocorrelation / partial autocorrelation (ACF/PACF):** measure how a series correlates with its own past values — used to choose lag features and to size ARIMA-style models.
- **Survivorship bias:** testing only on assets that still exist today silently excludes the failures, inflating backtest results.

5. Ensemble Methods & Model Evaluation

5.1 Ensembling

Method	Idea	Reduces
Bagging (Bootstrap Aggregating)	Train many models in parallel on random resampled subsets, then average/vote	Variance (overfitting)
Boosting	Train models sequentially, each correcting the previous one's errors	Bias (underfitting)
Stacking	Train a meta-model on top of several base models' outputs	Combines strengths of diverse model types
Weighted voting (this project's scoring engine)	Each "model" (strategy) casts a weighted vote; sum and threshold	Single-signal noise/false positives

Meta-labeling (a specific, well-regarded quant technique, from Marcos López de Prado's work): keep a primary model/rule for direction, then train a secondary classifier only to answer "will this signal actually work?" — its probability is used to size or filter trades rather than override direction. This is the recommended next step for this project's engine.

5.2 Evaluation metrics — classification

- **Confusion matrix:** TP, FP, TN, FN — the source of every other metric.
- **Precision** = $TP/(TP+FP)$ — "of what I flagged positive, how much was right" (minimizes false alarms). **Recall** = $TP/(TP+FN)$ — "of all actual positives, how many did I catch" (minimizes misses). There's always a trade-off; the right balance depends on the cost of each error type.
- **F1 score** = harmonic mean of precision and recall — a single number when both matter.
- **ROC curve / AUC:** true-positive rate vs false-positive rate across all thresholds; AUC = probability a random positive is ranked above a random negative. Misleading under heavy class imbalance — prefer **Precision-Recall AUC** there.
- **Calibration:** does a model's "70% confident" actually happen 70% of the time? Checked with reliability diagrams / Brier score — critical when a probability is used to size a bet (like meta-labeling or this project's confidence score).

5.3 Why accuracy is often the wrong metric

Under class imbalance (rare events) or asymmetric costs (a missed cancer diagnosis $\neq$ a false alarm; a losing trade $\neq$ a winning one), a model that always predicts the majority class can have high accuracy and zero value. Use precision/recall/F1, cost-weighted metrics, or in trading, expected value per decision and risk-adjusted return (Sharpe ratio, max drawdown) instead.

6. Deep Learning Fundamentals

6.1 The neural network basics

A neural network is layers of neurons: `output = activation(W·x + b)`. Stacking layers lets the network approximate arbitrarily complex functions (the "universal approximation theorem"). **Backpropagation** computes the gradient of the loss with respect to every weight by applying the chain rule backward through the network, so gradient descent can update every layer.

6.2 Activation functions

- **Sigmoid**: squashes to (0,1); suffers vanishing gradients in deep nets; still used at output for binary probabilities.
- **Tanh**: squashes to (-1,1), zero-centered, same vanishing-gradient issue.
- **ReLU**: $\max(0, x)$ — cheap, mitigates vanishing gradients, default for hidden layers; can "die" (stuck at zero gradient) — **Leaky ReLU / GELU** address that.
- **Softmax**: turns a vector of scores into a probability distribution — standard multi-class output layer, and the mechanism behind attention weights (§6.4) and LLM next-token sampling.

6.3 Training stabilizers

- **Batch normalization**: normalizes layer inputs per mini-batch, stabilizing and speeding up training, allowing higher learning rates.
- **Dropout**: randomly zeroes a fraction of neurons during training, forcing redundancy — prevents co-adapted overfitting; the classic regularizer for neural nets.
- **Vanishing / exploding gradients**: in deep or recurrent nets, repeated multiplication through the chain rule can shrink gradients to ~ 0 or blow them up; mitigated by ReLU, residual/skip connections, careful initialization, and gradient clipping.

6.4 Sequence models — RNN → LSTM → Transformer

RNNs process sequences step by step, carrying a hidden state forward — but struggle with long-range dependencies due to vanishing gradients. **LSTMs** (Long Short-Term Memory) add gated memory cells (input/forget/output gates) that let gradients flow across many steps, fixing that. **Transformers** (2017, "Attention Is All You Need") discard recurrence entirely: every position attends directly to every other position in parallel via **self-attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Query/Key/Value are learned linear projections of the input; the $\text{softmax}(QK^T)$ term produces attention weights — "how much should this token look at that token" — applied to V. **Multi-head attention** runs several of these in parallel with different learned projections, letting the model attend to different relationship types at once (syntax, coreference, etc.). Because there's no recurrence, transformers need **positional encoding** added to the input embeddings to know token order, and they parallelize far better than RNNs — the key reason they scaled to today's LLMs.

6.5 CNNs (brief — less central to this project's domain)

Convolutional layers slide small learned filters across the input, exploiting spatial locality and weight sharing — the backbone of image models, and occasionally applied to price charts as images or 1D convolutions over time series (less common than transformers/trees for tabular/finance data).

7. NLP, LLMs, RAG & Agents (the section most relevant to this project)

7.1 From words to embeddings

Tokenization: splitting text into model units. Modern LLMs use subword tokenization (**BPE** — Byte-Pair Encoding, or SentencePiece) — merges frequent character pairs iteratively, so rare words split into familiar subpieces instead of an "unknown" token. **Embeddings:** dense vectors representing meaning, positioned so that semantically similar items are close in vector space (cosine similarity). **word2vec/GloVe** were the earlier static-embedding generation; transformer embeddings are contextual (the same word gets a different vector depending on context).

7.2 The LLM training pipeline

1. **Pretraining:** self-supervised next-token prediction over huge unlabeled text corpora — where the model learns language and world knowledge.
2. **Supervised fine-tuning (SFT):** trained further on curated instruction/response examples to follow instructions.
3. **RLHF (Reinforcement Learning from Human Feedback):** a reward model is trained on human preference rankings of outputs; the LLM is then optimized (classically via **PPO**) against that reward model to prefer helpful/safe responses. **DPO** (Direct Preference Optimization) is a newer alternative that skips the separate reward model.

7.3 Prompting & inference-time control

- **Zero-shot / few-shot prompting:** asking directly, or providing a couple of worked examples in the prompt, with no weight updates — the model generalizes from pretraining alone.
- **Chain-of-thought prompting:** asking the model to reason step by step before answering, measurably improving performance on multi-step problems.
- **Temperature:** scales the logits before softmax; low temperature → more deterministic/greedy, high → more random/creative. This project sets temperature 0.3 for the chat — favoring faithful explanation over creativity.
- **Top-k / top-p (nucleus) sampling:** restrict sampling to the k most likely tokens, or the smallest set whose cumulative probability exceeds p — both trim the long tail of unlikely tokens.
- **Context window:** the maximum tokens (prompt + history + output) a model can attend to in one call; managing it (e.g. this project's 8-turn history cap, compacted evidence JSON) is a real engineering constraint, not just a prompting nicety.

7.4 RAG — Retrieval-Augmented Generation

Instead of relying purely on what the model memorized during pretraining (which goes stale and invites hallucination), fetch relevant, current, authoritative text at query time and hand it to the model as context, instructing it to answer from that context. Full pipeline: chunk documents → embed chunks → store in a **vector database** (FAISS, Chroma, Pinecone, pgvector) → embed the incoming query → retrieve nearest chunks by similarity (cosine/dot-product/Euclidean) → optionally **re-rank** → generate with the retrieved text injected into the prompt. **Chunking strategy** (fixed-size vs semantic/sentence-aware, with overlap) and **retrieval evaluation** (recall@k, MRR) are the two most commonly probed sub-topics. This project's chat is a minimal RAG variant where the "retrieval" is a computed evidence object rather than a search.

7.5 Hallucination — causes and mitigations

Cause: LLMs are next-token predictors optimized for plausibility, not truth; with no grounding they'll generate fluent, confident, and sometimes fabricated specifics. Mitigations: RAG/grounding (supply the facts), lower temperature, explicit "answer only from the provided context / say 'I don't know' otherwise" instructions, output citations, and a verification/self-consistency pass (sample multiple times, check agreement). No technique eliminates it completely — every production system needs a fallback path for when generation goes wrong, which is why this project keeps a non-LLM rule-based answer as its floor.

7.6 Structured output / constrained generation

Forcing a model's output to satisfy a schema (JSON Schema / Pydantic model / regex/grammar), via either prompting + validation-with-retry, or true constrained decoding that masks invalid tokens at sampling time. Converts a text generator into a typed, programmable component — essential once an LLM's output feeds downstream code rather than a human reader.

7.7 Agents, function calling & multi-agent systems

Function/tool calling: the model is given schemas of available functions; instead of prose it emits a structured call (name + arguments); the caller executes it and returns the result as a new message; the model continues, possibly calling more tools, until it produces a final answer. **ReAct** (Reason+Act) is the classic pattern name for interleaving reasoning traces with tool calls. **Multi-agent systems:** multiple LLM instances with different roles/personas (e.g. a bull analyst, bear analyst, and judge) collaborate or debate — shown to reduce single-pass errors and hallucination versus one model reasoning alone. **Memory:** short-term (conversation history in context) vs long-term (stored externally, retrieved via RAG) — this project uses short-term memory (last 8 turns) per symbol.

7.8 Fine-tuning vs RAG vs prompting — when to use which

Approach	Best for	Cost/complexity
Prompting (zero/few-shot)	Fast iteration, general tasks, small context needs	Lowest — no training
RAG	Needs current/proprietary/large knowledge; must reduce hallucination; wants citations	Medium — infra (vector DB) but no model training
Fine-tuning (full or LoRA/PEFT)	Needs a specific style/format baked in, domain jargon, or latency/cost gains from a smaller specialized model	Highest — needs labeled data + training/eval cycle

LoRA (Low-Rank Adaptation) and other **PEFT** (Parameter-Efficient Fine-Tuning) methods freeze the base model and train small low-rank adapter matrices instead — cuts fine-tuning cost by orders of magnitude versus updating all weights, and is why fine-tuning is accessible even without massive compute.

7.9 Model compression & efficient inference

Quantization: storing/computing weights in lower precision (e.g. FP16, INT8, INT4) to shrink memory and speed up inference with a small accuracy cost — how large models run on consumer hardware (e.g. local Ollama models in this project's design space). **Distillation:** training a smaller "student" model to mimic a larger "teacher"'s outputs. **Pruning:** removing low-importance weights/neurons. All three trade a small quality loss for large efficiency gains.

8. Statistics & Experimentation

8.1 Core statistical inference

- **Hypothesis testing:** null hypothesis (no effect) vs alternative; a **p-value** is the probability of seeing data this extreme *if the null were true* — not the probability the null is true (a very common misstatement to avoid in an interview).
- **Confidence intervals:** a range that would contain the true parameter in, e.g., 95% of repeated samples — communicates uncertainty that a point estimate hides.
- **Type I error** (false positive, rejecting a true null) vs **Type II error** (false negative, failing to reject a false null); **statistical power** = $1 - \text{Type II error rate}$.
- **Central Limit Theorem:** the sampling distribution of a mean tends toward normal as sample size grows, regardless of the underlying distribution — why normal-based tests are so broadly used.

8.2 A/B testing

Randomly split users/samples into control and treatment, measure a pre-registered metric, test for a significant difference. Pitfalls to name in interviews: **peeking** (checking significance repeatedly inflates false-positive rate — use sequential testing methods if you must), novelty effects, insufficient sample size/power, and multiple-comparisons inflation when testing many metrics at once (mitigated by Bonferroni or false-discovery-rate corrections).

8.3 Correlation, causation & leakage

Correlation $\neq$ causation — confounders can create spurious correlation. Proper causal claims need randomized experiments or causal-inference techniques (instrumental variables, diff-in-diff, propensity score matching). **Data leakage** — the ML-specific cousin of this idea — is when information from outside the legitimate training set (often, indirectly, the future or the label itself) leaks into features, producing inflated validation metrics that collapse in production.

8.4 Distributions worth recognizing

Normal (bell curve, CLT), Bernoulli/Binomial (success/failure counts), Poisson (rare event counts over time), and for finance specifically: returns are closer to **fat-tailed** distributions than normal (real markets have more extreme moves than a normal distribution predicts) — a key reason naive Gaussian risk models (like early Value-at-Risk methods) underestimated crash risk historically.

9. MLOps & Production ML Systems

9.1 The ML lifecycle

Data collection → feature engineering → training → evaluation → deployment → monitoring → retraining, as a loop, not a line. Most of the real-world engineering effort in ML systems is in the loop's plumbing, not the model itself — "hidden technical debt in ML systems" is the famous framing (Sculley et al., Google, 2015).

9.2 Key MLOps concepts

- **Experiment tracking** (MLflow, Weights & Biases): logging hyperparameters, metrics, and artifacts for every training run so results are reproducible and comparable.
- **Data/model versioning** (DVC, model registries): treating datasets and trained models as versioned artifacts, not files that quietly change.
- **Feature stores**: a central, consistent place to compute and serve features so training and serving don't drift apart ("training-serving skew").
- **CI/CD for ML**: automated tests for data schema, model quality gates, and staged rollout (shadow deployment, canary release) before full production traffic.
- **Drift: data drift** (input distribution changes), **concept drift** (the relationship between X and y itself changes — e.g. a market regime change), and **label/feedback drift** (delayed or biased labels). Monitored via statistical distance metrics (PSI, KL divergence) on live vs training distributions, and by tracking live performance against a baseline — exactly what this project's flag journal does, just with a 5–20 day delayed "label" (the forward return).
- **Shadow deployment**: run a new model alongside the current one on live traffic without acting on its output, to compare before cutover — this project's "paper trading before real money" staged rollout is the same idea one level up (strategy-level, not model-level).

10. Reinforcement Learning — The Basics

An **agent** takes **actions** in an **environment**, observing a **state** and receiving a **reward**; the goal is to learn a **policy** $\pi(a|s)$ that maximizes expected cumulative (often discounted) reward. Formalized as a **Markov Decision Process** (MDP): states, actions, transition probabilities, rewards, and a discount factor γ — the Markov property means the next state depends only on the current state and action, not the full history.

- **Value-based methods** (Q-learning, DQN): learn the expected return of each (state, action) pair, act greedily/ ϵ -greedily with respect to it.
- **Policy-based methods** (REINFORCE, PPO): directly learn/optimize the policy function; PPO (Proximal Policy Optimization) constrains each update to avoid destructively large policy shifts — the standard choice for both game-playing agents and LLM RLHF fine-tuning.
- **Exploration vs exploitation**: the agent must balance trying new actions (to discover better strategies) against repeating known-good ones — ϵ -greedy and upper-confidence-bound strategies are classic solutions, and this exact tension is why RL applied to a single stock's history (this project's proposed optimization #9) is prone to overfitting a small, non-stationary dataset.

11. Academic Q&A Bank (90+ Questions)

Pure conceptual/theory questions — the kind asked regardless of what any specific candidate has built. Answers are compressed flashcards; expand each aloud with an example when you practice.

A. Machine Learning Fundamentals (1–14)

1. Explain the bias-variance tradeoff.

Bias = error from an overly simple model (underfits); variance = error from over-sensitivity to the training sample (overfits). Total error $\approx \text{bias}^2 + \text{variance} + \text{irreducible noise}$; increasing model complexity trades one for the other, so the goal is the point that minimizes their sum on unseen data.

2. What is overfitting and how do you prevent it?

The model fits noise in the training data, not just the signal, so training performance is much better than validation performance. Prevent via more data, regularization (L1/L2/dropout), simpler models, early stopping, cross-validation, and ensembling.

3. Explain L1 vs L2 regularization.

L1 adds $\sum |w|$ to the loss — creates sparsity, some weights hit exactly zero (feature selection). L2 adds $\sum w^2$ — shrinks weights smoothly toward zero without eliminating them, handles correlated features better.

4. Why is k-fold cross-validation invalid for time-series data?

Random folds let the model train on future data and validate on past data, leaking information backward in time and inflating scores. Use walk-forward/rolling-origin validation instead — always train on strictly earlier data than you test on.

5. Bagging vs boosting — what's the difference?

Bagging trains many models in parallel on bootstrapped samples and averages them, reducing variance (e.g. Random Forest). Boosting trains models sequentially, each fitting the previous ensemble's residual errors, reducing bias (e.g. XGBoost, AdaBoost).

6. Explain precision, recall, and F1. When would you optimize for one over another?

Precision = $TP/(TP+FP)$, how trustworthy positive predictions are. Recall = $TP/(TP+FN)$, how many actual positives are caught. F1 is their harmonic mean. Optimize precision when false positives are costly (spam filters); recall when false negatives are costly (disease screening, fraud detection).

7. What is the ROC curve and what does AUC mean?

ROC plots true-positive rate against false-positive rate across all classification thresholds. AUC is the probability a randomly chosen positive example ranks above a randomly chosen negative one; 0.5 = random, 1.0 = perfect. It's threshold-independent but can look optimistic under severe class imbalance — use PR-AUC there.

8. Explain the curse of dimensionality.

As feature count grows, the volume of the space grows exponentially, so data becomes sparse and distance-based notions (like nearest neighbors) become less meaningful — most points end up nearly equidistant. Mitigated by dimensionality reduction (PCA) or feature selection.

9. How does a decision tree decide where to split?

It picks the feature and threshold that most reduces impurity — Gini impurity or entropy/information gain for classification, variance reduction for regression — greedily, at each node.

10. What is gradient boosting, mechanically?

Fit an initial weak model; compute residual errors; fit the next tree to predict those residuals; add it (scaled by a learning rate) to the ensemble; repeat. Each new tree corrects what the ensemble so far got wrong.

11. Explain feature scaling — why and when is it needed?

Algorithms that use distance or gradient magnitude (k-NN, SVM, linear/logistic regression, neural nets) are sensitive to feature scale; standardizing (zero mean, unit variance) or min-max scaling prevents larger-magnitude features from dominating. Tree-based models are scale-invariant and don't need it.

12. What is class imbalance and how do you handle it?

When one class vastly outnumbers another (fraud, rare disease, rare profitable trades), naive accuracy is misleading and models default to predicting the majority class. Handle via resampling (SMOTE, undersampling), class-weighted loss, threshold tuning, or metrics designed for imbalance (PR-AUC, F1, cost-sensitive evaluation).

13. Explain the difference between generative and discriminative models.

Discriminative models learn $P(y|x)$ directly — the decision boundary (logistic regression, SVM). Generative models learn $P(x|y)$ and $P(y)$, modeling how the data is generated, and can produce new samples (Naive Bayes, GANs, diffusion models, and LLMs in the sense that they model the full distribution of text).

14. What's the difference between parametric and non-parametric models?

Parametric models assume a fixed functional form with a fixed number of parameters regardless of data size (linear regression). Non-parametric models grow in complexity with the data (k-NN, decision trees) — more flexible, but need more data and are more prone to overfitting.

B. Deep Learning (15–28)

15. Explain backpropagation.

Using the chain rule to compute the gradient of the loss with respect to every weight, propagating error backward from the output layer through each hidden layer, so gradient descent can update all parameters simultaneously.

16. What causes vanishing/exploding gradients and how is it fixed?

Repeated multiplication of small (or large) derivatives through many layers/timesteps shrinks (or blows up) the gradient. Fixed via ReLU activations, residual/skip connections, careful weight initialization, gradient clipping, and architectures like LSTM designed to preserve gradient flow.

17. Why is ReLU preferred over sigmoid in hidden layers?

Sigmoid saturates (near-zero gradient) for large $|x|$, causing vanishing gradients in deep networks; it's also not zero-centered. ReLU's gradient is constant (1) for positive inputs, keeping gradients flowing, and is cheaper to compute — at the cost of "dead neurons" for negative inputs, addressed by Leaky ReLU/GELU.

18. What is dropout and why does it work?

Randomly zeroing a fraction of neurons each training step, forcing the network to not rely on any single neuron/co-adapted group — approximates training an ensemble of thinner networks and averaging them at test time, reducing overfitting.

19. Explain batch normalization.

Normalizes each layer's inputs to zero mean/unit variance per mini-batch (with learned scale/shift), reducing internal covariate shift, stabilizing training, and allowing higher learning rates.

20. Why did transformers replace RNNs for most sequence tasks?

RNNs process tokens sequentially (slow, hard to parallelize) and struggle with long-range dependencies despite LSTM gating. Transformers use self-attention so every token attends to every other token directly and in parallel, capturing long-range dependencies more effectively and training far faster on modern hardware.

21. Explain self-attention in your own words, with the formula.

Each token produces a Query, Key, and Value vector (learned projections). $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$: the dot product of Q and K measures relevance between token pairs, softmax turns that into weights, and those weights combine the Value vectors — so each token's new representation is a relevance-weighted blend of every other token's information.

22. Why divide by $\sqrt{d_k}$ in attention?

Without scaling, dot products grow large in magnitude as dimension d_k increases, pushing softmax into regions with extremely small gradients (saturation). Dividing by $\sqrt{d_k}$ keeps the variance of the dot products stable regardless of dimension.

23. What is multi-head attention?

Running several attention operations in parallel, each with its own learned Q/K/V projections, then concatenating results. Lets the model attend to different types of relationships (e.g. syntax vs coreference) simultaneously, rather than one averaged attention pattern.

24. Why do transformers need positional encoding?

Self-attention itself is permutation-invariant — it has no built-in notion of token order. Positional encodings (sinusoidal or learned) are added to token embeddings so the model can use sequence position.

25. What is transfer learning and why is it powerful?

Taking a model pretrained on a large general dataset and adapting it to a specific task with much less data/compute (feature extraction or fine-tuning), because the pretrained model already learned broadly useful representations. This is how BERT/FinBERT-style models and most modern vision/NLP systems are used in practice.

26. Explain the vanishing gradient problem specifically in RNNs, and how LSTM fixes it.

Repeated multiplication by the same recurrent weight matrix across many timesteps shrinks gradients exponentially, so RNNs "forget" long-range context. LSTM introduces a memory cell with input/forget/ output gates that allow gradients to flow largely unchanged across timesteps when the forget gate is open, preserving long-range signal.

27. What's the difference between a loss function and an evaluation metric?

The loss function must be differentiable (or at least optimizable) since it directly drives gradient updates during training (e.g. cross-entropy, MSE). Evaluation metrics measure what you actually care about and need not be differentiable (e.g. accuracy, F1, Sharpe ratio) — sometimes they're the same, often they aren't.

28. What is weight initialization and why does it matter?

Poor initialization (all zeros, or too large/small) can cause symmetric neurons that never differentiate, or immediate vanishing/exploding activations. Xavier/Glorot and He initialization scale initial weights based on layer fan-in/fan-out to keep activations and gradients in a stable range at the start of training.

C. NLP & Large Language Models (29–48)

29. What is tokenization and why do LLMs use subword tokenization (BPE)?

Splitting text into model-readable units. Subword methods like Byte-Pair Encoding iteratively merge frequent character/byte pairs, so common words stay whole while rare/unseen words decompose into familiar pieces — avoiding a fixed vocabulary's "unknown token" problem and keeping vocabulary size manageable.

30. What are embeddings, and how do they differ from one-hot encoding?

Embeddings are dense, low-dimensional vectors placing semantically similar items near each other in vector space, learned from data. One-hot vectors are sparse, high-dimensional, and carry no notion of similarity — the distance between any two one-hot vectors is identical regardless of meaning.

31. Explain the three stages of training a modern chat LLM.

(1) Pretraining: self-supervised next-token prediction on massive text corpora — learns language and world knowledge. (2) Supervised fine-tuning: trained on curated instruction-response pairs to follow instructions. (3) RLHF/DPO: aligned to human preference using a reward model + RL (or the more direct DPO method), optimizing for helpful, honest, safe responses.

32. What is RLHF and why is it needed if pretraining already works?

Pretraining optimizes for predicting plausible next text, not for being helpful/safe/aligned with what a human actually wants. RLHF trains a reward model on human preference comparisons between candidate responses, then fine-tunes the LLM (via PPO) to maximize that learned reward — directly targeting "humans prefer this response" rather than "this text is statistically likely."

33. What is temperature in LLM sampling, and what does temperature=0 mean?

Temperature scales the logits before the softmax that turns them into a probability distribution over the vocabulary. Low temperature sharpens the distribution toward the most likely tokens (more deterministic); high temperature flattens it (more random/diverse). Temperature→0 approaches greedy decoding — always picking the single most likely token.

34. Explain top-k vs top-p (nucleus) sampling.

Top-k restricts sampling to the k highest-probability tokens, discarding the rest. Top-p (nucleus) instead takes the smallest set of tokens whose cumulative probability exceeds p — adaptive to how confident the distribution is, unlike top-k's fixed cutoff count.

35. What causes LLM hallucination and how do you mitigate it?

LLMs are trained to produce statistically plausible continuations, not verified facts, so with no grounding they can generate fluent but false specifics — worse for long-tail facts, recent events, or precise numbers. Mitigate with RAG/grounding, explicit "answer only from provided context" instructions, low temperature, citation requirements, and self-consistency checks — no single technique eliminates it.

36. Explain Retrieval-Augmented Generation end to end.

Offline: chunk source documents, embed each chunk, store vectors in a vector database. Online: embed the incoming query, retrieve the most similar chunks (cosine/dot-product similarity, optionally re-ranked), inject them into the prompt as context, and generate an answer instructed to rely on that context — reducing hallucination and allowing up-to-date/proprietary knowledge without retraining the model.

37. How would you choose a chunk size and overlap for a RAG pipeline?

Small chunks improve retrieval precision but may lose surrounding context; large chunks preserve context but dilute relevance and cost more tokens. A common starting point is a few hundred tokens per chunk with ~10–20% overlap between chunks so information near a boundary isn't split away from its context; tune empirically against a retrieval-quality metric like recall@k.

38. How do you evaluate a RAG system's retrieval quality?

Recall@k (is the relevant chunk in the top k retrieved?), Mean Reciprocal Rank (how high does the first relevant result rank?), and end-to-end answer quality/faithfulness (does the generated answer actually match the retrieved context, checked manually or with an LLM-as-judge).

39. What is function/tool calling in LLM agents?

The model is given structured schemas (name, description, parameters) of available functions. Instead of free text, it can emit a structured call requesting a specific function with specific arguments; the calling code executes it and feeds the result back as a new message, letting the model incorporate real, up-to-date, or computed information it couldn't otherwise access.

40. What is the ReAct pattern?

Interleaving explicit reasoning steps ("Thought: I need X") with actions (tool calls) and observations (tool results) in a loop, rather than either pure chain-of-thought reasoning or pure tool-calling alone — shown to improve both reliability and interpretability of agent behavior.

41. What are structured outputs / constrained decoding, and why do they matter?

Forcing a model's output to conform to a schema (JSON Schema, regex, grammar) — either via prompting-plus-validation-with-retry, or true constrained decoding that masks invalid next-tokens during sampling. Converts free-text generation into a reliable, typed interface that downstream code can consume without brittle parsing.

42. Fine-tuning vs RAG vs prompting — how do you decide?

Prompting first (cheapest, fastest iteration). Add RAG when the model needs current, proprietary, or large volumes of knowledge and you want hallucination reduction with citations. Fine-tune when you need a specific output style/format baked in, domain-specific behavior prompting can't reliably achieve, or cost/latency gains from a smaller specialized model — and only once you have enough quality labeled examples.

43. What is LoRA and why is it useful?

Low-Rank Adaptation: freeze the pretrained model's weights and inject small trainable low-rank matrices into specific layers, training only those. Achieves fine-tuning-level task adaptation while updating a tiny fraction of parameters — massively cheaper in compute and storage, and multiple LoRA adapters can be swapped in/out of the same base model.

44. What is quantization and what's the trade-off?

Representing model weights/activations with fewer bits (FP16, INT8, INT4 instead of FP32), shrinking memory footprint and speeding up inference, at a small, usually acceptable, cost to output quality/precision — the key enabler of running large models on consumer hardware.

45. How would you evaluate an LLM-powered feature before shipping it?

Build a held-out set of representative queries with reference answers or rubrics; measure task-specific metrics (accuracy, faithfulness to source, refusal rate on out-of-scope questions); use LLM-as-judge or human review for qualities hard to measure automatically (helpfulness, tone); and test failure modes explicitly — adversarial prompts, missing context, rate-limit/error handling.

46. What is in-context learning?

An LLM's ability to adapt its behavior based purely on examples/instructions given in the prompt at inference time, with no gradient update to the model's weights — the mechanism behind zero-shot and few-shot prompting.

47. What is the difference between a base model and an instruction-tuned/chat model?

A base model is the raw pretrained next-token predictor — it completes text but doesn't reliably follow instructions or converse naturally. An instruction-tuned/chat model has additional supervised fine-tuning (and usually RLHF/DPO) specifically teaching it to follow instructions and hold a helpful, safe dialogue.

48. What is a system prompt and how does it differ from a user message?

The system prompt sets persistent instructions/context/persona for the whole conversation (role, constraints, grounding data) and is typically weighted more authoritatively by the model than user turns; user/assistant messages are the actual back-and-forth dialogue the system prompt governs.

D. Statistics & Experimentation (49–58)

49. What does a p-value actually mean?

The probability of observing data at least as extreme as what you saw, *assuming the null hypothesis is true* — it is NOT the probability that the null hypothesis is true, a very common misinterpretation.

50. Type I vs Type II error?

Type I: rejecting a true null hypothesis (false positive). Type II: failing to reject a false null hypothesis (false negative). They trade off against each other via the significance threshold; statistical power = $1 - P(\text{Type II error})$.

51. What is the Central Limit Theorem and why does it matter?

The distribution of sample means approaches a normal distribution as sample size grows, regardless of the population's underlying distribution — the theoretical justification for using normal-distribution-based confidence intervals and hypothesis tests so broadly.

52. How would you design an A/B test, and what commonly goes wrong?

Define a single primary metric and hypothesis before starting, calculate required sample size for adequate power, randomize assignment, run for a pre-set duration/sample size without peeking early (peeking inflates false-positive rate), and correct for multiple comparisons if testing several metrics.

53. What is data leakage and give an example.

Information that wouldn't be legitimately available at prediction time ends up in the training features, inflating validation performance in a way that collapses in production. Example: using a stock's closing price on day t to predict something also derived from day t 's close, or normalizing features using statistics computed over the whole dataset including future/test data.

54. Explain the difference between correlation and causation with an example.

Two variables can move together without one causing the other, often due to a confounding variable. Example: ice cream sales and drowning deaths correlate — both are driven by hot weather, not each other.

55. What is survivorship bias?

Drawing conclusions only from entities/data that "survived" some selection process while ignoring those that didn't — e.g. backtesting a strategy only on companies that still exist today ignores delisted/bankrupt companies, inflating apparent historical returns.

56. What's the difference between a confidence interval and a prediction interval?

A confidence interval estimates uncertainty around a population parameter (e.g. the mean). A prediction interval estimates the range for a single new observation, which must also account for the individual data point's own variance — so it's always wider than the corresponding confidence interval.

57. What is multicollinearity and why is it a problem?

When predictor variables are highly correlated with each other, coefficient estimates in linear models become unstable and hard to interpret (small data changes cause large coefficient swings), though prediction accuracy is often still fine. Detected via correlation matrices or Variance Inflation Factor; addressed via removing/combining features, PCA, or L2 regularization.

58. Explain stationarity and why it matters for time-series modeling.

A stationary series has statistical properties (mean, variance, autocorrelation) that don't change over time. Most classical time-series models (ARIMA) assume stationarity; raw price series usually trend and aren't stationary, so practitioners model returns or differenced series instead, checking with tests like the Augmented Dickey-Fuller test.

E. MLOps & Production Systems (59–68)

59. What is model/data drift, and how do you detect it in production?

Data drift: the input feature distribution shifts from training time. Concept drift: the relationship between inputs and the true output changes (the same X now implies a different y). Detected by comparing live feature/prediction distributions against training-time baselines (PSI, KL divergence) and by monitoring live performance against ground truth once labels arrive.

60. What is training-serving skew and how do you prevent it?

When features are computed differently (different code paths, different data freshness) at training time versus serving time, causing a production model to behave unexpectedly despite good offline metrics. Prevented by sharing the exact same feature computation code/pipeline (often via a feature store) for both training and serving.

61. What is a feature store and why use one?

A centralized system for defining, computing, storing, and serving features consistently across training and online inference, avoiding duplicated/divergent feature logic and enabling feature reuse across teams/models.

62. What is shadow deployment / canary release?

Shadow deployment runs a new model on live traffic in parallel with the current model, without acting on its output, purely to compare performance before cutover. Canary release routes a small percentage of real traffic to the new model's actual decisions, gradually increasing if it performs well — both de-risk a full rollout.

63. How do you monitor a deployed ML model over time?

Track input distribution drift, prediction distribution shifts, latency/error rates, and — once ground truth becomes available — live accuracy/business metrics against a baseline, with alerting thresholds and a rollback/retraining trigger when metrics degrade.

64. What is experiment tracking and why does it matter?

Systematically logging every training run's hyperparameters, code version, data version, and resulting metrics (tools: MLflow, Weights & Biases) so results are reproducible, comparable, and auditable — without it, "which config produced our best model" becomes unanswerable at scale.

65. Explain the difference between online and offline evaluation.

Offline evaluation uses historical/held-out data before deployment (fast, cheap, but can miss real-world effects like user behavior changes). Online evaluation (A/B tests, shadow/canary deployments) measures actual live impact but is slower, riskier, and requires real traffic.

66. What is a rollback strategy for ML models and why is it needed?

The ability to quickly revert to a previous known-good model version if a newly deployed model underperforms or misbehaves in production — requires model versioning/registries and monitoring sensitive enough to trigger the rollback before serious damage.

67. How would you version datasets for ML reproducibility?

Tools like DVC (Data Version Control) or dataset hashing/snapshotting so that a given experiment can be tied to an exact, immutable version of its training data, not just the code — since data changes are often the real cause of a model's behavior changing.

68. What's the difference between batch and real-time (online) inference?

Batch inference precomputes predictions for a large set of inputs on a schedule (cheaper, higher throughput, tolerates latency). Real-time inference serves predictions per-request with low latency requirements (more complex infrastructure, needs to handle traffic spikes and be fast per call).

F. Reinforcement Learning (69–75)

69. What is a Markov Decision Process?

A formalism for sequential decision-making: a set of states, actions, transition probabilities $P(s'|s,a)$, a reward function, and a discount factor γ — the Markov property means the next state/reward depends only on the current state and action, not the history that led there.

70. Explain the exploration-exploitation trade-off.

An agent must balance trying new/uncertain actions to discover potentially better strategies (exploration) against repeating actions already known to work well (exploitation). Too much exploitation risks getting stuck in a suboptimal policy; too much exploration wastes reward on known-bad actions.

71. Value-based vs policy-based RL methods?

Value-based methods (Q-learning, DQN) learn the expected return of each state-action pair and act greedily with respect to it. Policy-based methods (REINFORCE, PPO) directly parameterize and optimize the action-selection policy itself, which handles continuous action spaces more naturally and can learn stochastic policies.

72. What is PPO and why is it widely used?

Proximal Policy Optimization optimizes the policy while constraining each update to stay close to the previous policy (via a clipped objective), preventing the destructively large policy swings that plagued earlier policy gradient methods — a good balance of sample efficiency and training stability, and the standard choice for both robotics/game RL and LLM RLHF fine-tuning.

73. What is the discount factor γ and what happens at its extremes?

It weights how much future rewards matter versus immediate ones. $\gamma \rightarrow 0$ makes the agent myopic (only immediate reward matters); $\gamma \rightarrow 1$ makes it weight distant future rewards almost as much as immediate ones (can slow convergence and increase variance in estimates).

74. Why is applying RL to a single financial time series prone to overfitting?

A single asset's history is one (short, non-stationary, noisy) sample path, not a distribution of independent episodes to learn robust general behavior from — the agent can easily learn to exploit quirks specific to that historical sample that won't recur, unlike a game environment that can be simulated for millions of independent episodes.

75. What is reward shaping and what risk does it carry?

Designing intermediate rewards to guide learning faster toward a sparse/delayed true objective. Risk: a poorly designed shaped reward can be "gamed" — the agent maximizes the proxy reward in an unintended way that doesn't actually achieve the real goal (reward hacking).

G. Applied / Finance-Adjacent ML (76–92)

76. What is meta-labeling and why is it useful in quantitative finance?

Keep a primary model/rule that decides direction (buy/sell), and train a separate secondary classifier only to predict whether that primary signal will actually succeed, using it to size or filter trades. It layers a learnable, evaluable component on top of an explainable base signal rather than replacing it, and directly targets a better precision/risk trade-off.

77. Why is backtested performance often worse live? Name the main causes.

Look-ahead bias, overfitting to historical noise via excessive tuning, ignoring transaction costs/slippage/taxes, survivorship bias in the asset universe, and regime change (the market conditions that produced the edge no longer hold).

78. Why might accuracy be a poor metric for a trading model but a fine one elsewhere?

Trading payoffs are asymmetric — a strategy right 45% of the time with big wins and small losses can be far more profitable than one right 70% of the time with small wins and big losses. Accuracy ignores payoff magnitude entirely; expected value and risk-adjusted return (Sharpe ratio, max drawdown) are the metrics that actually matter.

79. What is the Sharpe ratio?

$(\text{Mean portfolio return} - \text{risk-free rate}) / \text{standard deviation of returns}$ — return per unit of volatility risk taken. Higher is better; it penalizes volatile returns even if the average return is the same as a smoother strategy.

80. What is walk-forward optimization, specifically in trading strategy design?

Repeatedly: fit/tune parameters on an in-sample window, test unchanged on the next out-of-sample window, then roll both windows forward and repeat — simulating how a strategy would really be re-tuned and deployed over time, and directly guarding against overfitting a single historical period.

81. Explain ensemble weighting: fixed weights vs learned weights — trade-offs?

Fixed/hand-set weights (based on domain judgment) are simple, interpretable, and can't overfit to noise, but may not reflect true relative signal quality. Learned weights (fit via regression/optimization on historical performance) can capture real differences in signal value but risk overfitting, especially with limited data, and need the same walk-forward discipline as any other trained model.

82. What is regime detection and why does it matter for strategy performance?

Identifying which broad market condition (trending, ranging/choppy, high/low volatility, panic) currently holds, since many strategies' effectiveness is regime-dependent (e.g. mean-reversion loses money in strong trends). Detected via clustering, Hidden Markov Models, or volatility/trend indicators, then used to switch or re-weight strategies per regime.

83. What is an ATR (Average True Range) and what general statistical idea does it represent?

A volatility measure — the smoothed average of the true range (accounting for gaps) per bar. It's a domain-specific instance of the general idea of a rolling/exponentially-weighted moving measure of dispersion, used to normalize "how big is a significant move" relative to an asset's own typical behavior.

84. What is a z-score and why is a volatility/trend filter often applied alongside it?

$z\text{-score} = (\text{value} - \text{rolling mean}) / \text{rolling standard deviation}$ — measures how many standard deviations current price is from its recent average, used for mean-reversion signals. A trend filter (e.g. ADX) is applied because "stretched from the mean" behaves differently in a ranging market (reverts) versus a strongly trending one (keeps stretching) — the raw statistic alone conflates two different regimes.

85. Explain the difference between a leading and a lagging indicator, with examples.

Leading indicators attempt to predict future price movement before it happens (e.g. RSI divergence, order-flow signals) — more useful but noisier/less reliable. Lagging indicators confirm a move that's already underway (e.g. moving-average crossovers) — more reliable but later, giving up some of the move.

86. What is position sizing and why does it matter as much as signal quality?

How much capital to allocate per trade. Even a strategy with a real statistical edge can go broke with oversized bets (ruin risk) or underperform with undersized ones; methods like the Kelly criterion or volatility-targeted sizing size each bet by its edge and risk rather than a fixed amount, which can matter more for long-run compounding than marginally improving entry signal accuracy.

87. What is the Kelly criterion, conceptually?

A formula for the fraction of capital to bet that maximizes the long-run geometric growth rate of wealth, given a strategy's win probability and payoff ratio. Full Kelly is typically too aggressive/volatile in practice given estimation uncertainty, so practitioners commonly use a fraction of it ("half-Kelly").

88. What's the difference between supervised feature-based prediction and a rules-based/expert-system approach, and when would you pick each?

Rules-based systems encode human domain knowledge directly (explainable, no training data needed, deterministic, easy to debug and audit) but can't automatically discover interaction effects or adapt as patterns shift. Supervised ML can discover non-obvious patterns and combinations from data but needs sufficient quality labeled history, is more prone to overfitting, and is often less directly explainable. A common effective hybrid: rules/domain features as engineered inputs to a learned model that decides how to weight and combine them.

89. Explain what "false discovery" risk means when you test many trading strategies/features.

Testing many strategies or feature combinations against historical data means some will look profitable purely by chance (multiple-comparisons problem), even with no real edge — the more variations tried, the higher the chance of finding an accidental winner. Mitigated with out-of-sample holdout data never touched during search, correction for the number of trials, and requiring statistical/economic significance beyond a simple best-of-many result.

90. What is slippage and why does it matter for evaluating a strategy's real edge?

The difference between a trade's expected price (e.g. at signal time) and its actual executed price, caused by market movement between decision and execution, bid-ask spread, and order size relative to liquidity. A backtest ignoring slippage (and fees) can show a profitable edge that entirely disappears once realistic execution costs are included — a primary reason live results diverge from paper.

91. What is an anchored calculation (e.g. anchored VWAP) and what statistical concept does it use?

A cumulative statistic computed only from a specific starting point forward (an "anchor," e.g. a significant swing low) rather than over a fixed rolling window — conceptually a cumulative (expanding-window) weighted average rather than a rolling one, chosen so the calculation reflects "since this specific event" rather than an arbitrary fixed lookback.

92. How would you decide whether an observed strategy's outperformance is signal or noise?

Check statistical significance with an adequate sample size (dozens of trades minimum, ideally 100+), compare against a relevant baseline (buy-and-hold, not zero), verify performance isn't driven by one or two outlier trades, confirm results hold walk-forward out-of-sample rather than only in the tuned period, and check consistency across different assets/time periods rather than one lucky series.
